



BIG DATA

2025/26

Ciclo	Especialización IA & BIG DATA
Nombre	Rodrigo Medina
Correo	YMQ06518@educastur.es
Nº Unidad Didáctica	03

BIG DATA

- PR_03.1

1.	APARTADO A.....	2
2.	APARTADO B	8
3.	APARTADO C	20

1. APARTADO A

1. Muestra el total de hombres y mujeres que hay en el archivo u.user.

Cargar datos desde el archivo, todos los usuarios

```
usuarios = LOAD 'pr03_1/u.user' USING PigStorage('|') AS
(cod:int,edad:int,genero:chararray,rol:chararray,cp:chararray);
```

Agrupar registros por genero (M y F)

```
generos = GROUP usuarios BY genero;
```

Proyección realizando un conteo total de cada grupo

```
totalGeneros = FOREACH generos GENERATE group,
COUNT(usuarios);
```

Mostrar resultado

```
dump totalGeneros;
```

```
grunt> usuarios = LOAD 'pr03_1/u.user' USING PigStorage('|') AS (cod:int,
edad:int,genero:chararray,rol:chararray,cp:chararray);
grunt> generos = GROUP usuarios BY genero;
grunt> totalGeneros = FOREACH generos GENERATE group, COUNT(usuarios);
grunt> dump totalGeneros;
```

```
2025-11-12 17:53:49,901 [main] INFO org.apache.hadoop.mapreduce
.lib.input.FileInputFormat - Total input paths to process : 1
2025-11-12 17:53:49,901 [main] INFO org.apache.pig.backend.hado
op.executionengine.util.MapRedUtil - Total input paths to proces
s : 1
(F,273)
(M,670)
grunt> |
```



2. Mediante instrucciones de PIG encontrar las 10 ocupaciones más frecuentes entre los usuarios.

*A partir de ahora, se sobre entiende que el archivo ya estar cargado en usuarios

Agrupar por rol (ocupación)

```
roles= GROUP usuarios BY rol;
```

Contar el total de cada rol

```
totalRoles = FOREACH roles GENERATE group,  
COUNT(usuarios) AS total;
```

Ordenar de forma descendente los roles más repetidos

```
ordenarRoles = ORDER totalRoles BY total DESC;
```

Limitar el listado a las 10 primeras posiciones de los roles más habituales

```
topRoles = LIMIT ordenarRoles 10;
```

Mostrar resultado

```
dump topRoles;
```

```
grunt> roles = GROUP usuarios BY rol;  
grunt> totalRoles = FOREACH roles GENERATE group, COUNT(usuarios) AS total;  
grunt> ordenarRoles = ORDER totalRoles BY total DESC;  
grunt> topRoles = LIMIT ordenarRoles 10;  
grunt> dump topRoles;
```

```
ionengine.util.MapRedUtil - Total input paths to process : 1  
(student,196)  
(other,105)  
(educator,95)  
(administrator,79)  
(engineer,67)  
(programmer,66)  
(librarian,51)  
(writer,45)  
(executive,32)  
(scientist,31)  
grunt> |
```

3. Muestra la edad media por géneros.

Agrupar por géneros

```
generos = GROUP usuarios BY genero;
```

Calcular edad media de cada género

```
mediaGeneros = FOREACH generos GENERATE group,
AVG(usuarios.edad);
```

Mostrar resultado

```
dump mediaGeneros;
```

```
grunt> generos = GROUP usuarios BY genero;
grunt> mediaGeneros = FOREACH generos GENERATE group,AVG(usuarios.edad);
grunt> dump mediaGeneros;
```

```
2025-11-12 18:36:39,531 [main] INFO org.apache.pig.backend.hadoop.execut
ionengine.util.MapRedUtil - Total input paths to process : 1
(F,33.81318681318681)
(M,34.149253731343286) ←
```

4. Muestra la edad media por ocupaciones.

Agrupar por rol (ocupación)

```
roles= GROUP usuarios BY rol;
```

Calcular edad media por rol

```
mediaRoles = FOREACH roles GENERATE group,
AVG(usuarios.edad);
```

Mostrar resultado

```
dump mediaRoles;
```

BIG DATA

```
grunt> roles = GROUP usuarios BY rol;  
grunt> mediaRoles = FOREACH roles GENERATE group,AVG(usuarios.edad);  
grunt> dump mediaRoles;
```

```
(none,26.555555555555557)  
(other,34.523809523809526)  
(artist,31.392857142857142)  
(doctor,43.57142857142857)  
(lawyer,36.75)  
(writer,36.31111111111111)  
(retired,63.07142857142857)  
(student,22.081632653061224)  
(educator,42.01052631578948)  
(engineer,36.38805970149254)  
(salesman,35.666666666666664)  
(executive,38.71875)  
(homemaker,32.57142857142857)  
(librarian,40.0)  
(marketing,37.61538461538461)  
(scientist,35.54838709677419)  
(healthcare,41.5625)  
(programmer,33.121212121212125)  
(technician,33.148148148148145)  
(administrator,38.74683544303797)  
(entertainment,29.22222222222222)  
grunt> |
```

5. Guarda el resultado de las cuatro consultas anteriores en un script de extensión “.pig”. Ejecútalo. (recuerda, siempre en la carpeta /user/maría_dev)

Guardo en un archivo de texto plano todas las instrucciones anteriores. Lo copio mediante scp en Linux y lo ejecuto

```
script.pig  
Archivo  Editar  Ver  
  
-- Cargar datos dese el archivo, todos los usuarios  
usuarios = LOAD 'pr03_1/u.user' USING PigStorage('|') AS  
(cod:int,edad:int,genero:chararray,rol:chararray,cp:chararray);  
  
-- 1. Muestra el total de hombres y mujeres que hay en el archivo u.user.  
-- Agrupar registros por genero (M y F)  
generos = GROUP usuarios BY genero;  
  
-- Proyección realizando un conteo total de cada grupo  
totalGeneros = FOREACH generos GENERATE group, COUNT(usuarios);  
  
-- Mostrar resultado  
dump totalGeneros;
```

BIG DATA

pig script.pig

```
maria_dev@sandbox-hdp ~]$ pig script.pig
2025/11/13 18:32:35 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2025/11/13 18:32:35 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2025/11/13 18:32:35 INFO pig.ExecTypeProvider: Trying ExecType : TEZ_LOCAL
2025/11/13 18:32:35 INFO pig.ExecTypeProvider: Trying ExecType : TEZ
2025/11/13 18:32:35 INFO pig.ExecTypeProvider: Picked TEZ as the ExecType
2025-11-13 18:32:35,881 [main] INFO org.apache.pig.Main - Apache Pig version 0.16.0.2.6.5.0-292
tory) compiled May 11 2018, 07:56:28
```

De nuevo, se muestran los 4 resultados anteriores

```
2025-11-13 18:32:50,485 [main] INFO
hs to process : 1
(F,273)
(M,670)
```

```
2025-11-13 18:32:57,460 [main]
hs to process : 1
(F,33.81318681318681)
(M,34.149253731343286)
```

```
2025-11-13 18:32:55,109
hs to process : 1
(student,196)
(other,105)
(educator,95)
(administrator,79)
(engineer,67)
(programmer,66)
(librarian,51)
(writer,45)
(executive,32)
(scientist,31)
```

```
2025-11-13 18:33:00,796 [main] INFO
hs to process : 1
(none,26.555555555555557)
(other,34.523809523809526)
(artist,31.392857142857142)
(doctor,43.57142857142857)
(lawyer,36.75)
(writer,36.31111111111111)
(retired,63.07142857142857)
(student,22.081632653061224)
(educator,42.01052631578948)
(engineer,36.38805970149254)
(salesman,35.666666666666664)
(executive,38.71875)
(homemaker,32.57142857142857)
(librarian,40.0)
(marketing,37.61538461538461)
(scientist,35.54838709677419)
(healthcare,41.5625)
(programmer,33.121212121212125)
(technician,33.148148148148145)
(administrator,38.74683544303797)
(entertainment,29.222222222222222)
```

6. Almacena la salida de las cuatro consultas anteriores en una carpeta de HDFS llamada pig_usuarios

Puede haber dos formas de realizarlo.

- Añadiendo al script las líneas para almacenar los resultados (STORE)
- Desde pig, tras ejecutar los comandos y tener activos los alias almacenemos los resultados

BIG DATA

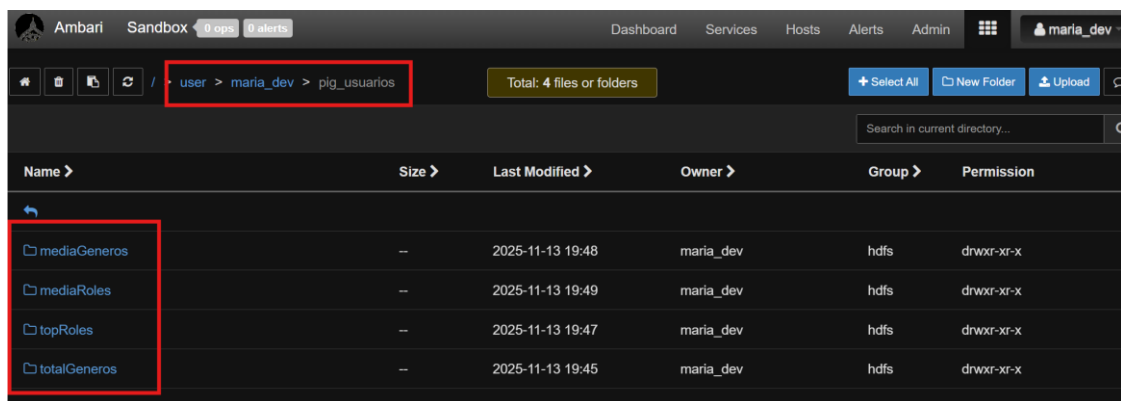
```
STORE totalGeneros INTO 'pig_usuarios/totalGeneros' USING  
PigStorage(',');
```

```
STORE topRoles INTO 'pig_usuarios/topRoles' USING  
PigStorage(',');
```

```
STORE mediaGeneros INTO 'pig_usuarios/mediaGeneros' USING  
PigStorage(',');
```

```
STORE mediaRoles INTO 'pig_usuarios/mediaRoles' USING  
PigStorage(',');
```

Una vez almacenados los resultados de los alias, podemos comprobar desde la interfaz de ambari el resultado.



Name >	Size >	Last Modified >	Owner >	Group >	Permission
mediaGeneros	—	2025-11-13 19:48	maria_dev	hdfs	drwxr-xr-x
mediaRoles	—	2025-11-13 19:49	maria_dev	hdfs	drwxr-xr-x
topRoles	—	2025-11-13 19:47	maria_dev	hdfs	drwxr-xr-x
totalGeneros	—	2025-11-13 19:45	maria_dev	hdfs	drwxr-xr-x

2. APARTADO B

1. Carga y descripción del dataset

- Carga el archivo (retail_sales_dataset.csv) usando PigStorage(',') y define un esquema correctamente para cada tipo de campo.

```
ventas = LOAD 'pr03_1/retail_sales_dataset.csv' USING
PigStorage(',') AS (ventaID:int, fecha:chararray,
clienteID:chararray, genero:chararray, edad:int,
categoria:chararray, cantidad:int, precioUnidad:double,
precioTotal:double);
```

```
grunt> ventas = LOAD 'pr03_1/retail_sales_dataset.csv' USING PigStorage(',') AS (ventaID:int,
fecha:chararray, clienteID:chararray, genero:chararray, edad:int, categoria:chararray, cantida
d:int, precioUnidad:double, precioTotal:double);
grunt> |
```

- Usa DESCRIBE para ver el esquema y DUMP para ver las primeras tuplas.

```
describe ventas;
```

```
grunt> describe ventas;
ventas: {ventaID: int, fecha: chararray, clienteID: chararray, genero: chararray, edad: int, catego
ria: chararray, cantidad: int, precioUnidad: double, precioTotal: double}
grunt>
```

```
dump ventas;
```

```
- Total input paths to process : 1
(,Date, Customer ID, Gender, , Product Category, , ,)
(1, 2023-11-24, CUST001, Male, 34, Beauty, 3, 50.0, 150.0)
(2, 2023-02-27, CUST002, Female, 26, Clothing, 2, 500.0, 1000.0)
(3, 2023-01-13, CUST003, Male, 50, Electronics, 1, 30.0, 30.0)
(4, 2023-05-21, CUST004, Male, 37, Clothing, 1, 500.0, 500.0)
(5, 2023-05-06, CUST005, Male, 30, Beauty, 2, 50.0, 100.0)
(6, 2023-04-25, CUST006, Female, 45, Beauty, 1, 30.0, 30.0)
```

BIG DATA

- Calcula cuántas transacciones totales tiene el dataset (COUNT).

```
ventasGrupo = GROUP ventas ALL;  
totalVentas = FOREACH ventasGrupo GENERATE group,  
COUNT(ventas.ventaID) AS total;
```

```
grunt> ventasGrupo = GROUP ventas ALL;  
grunt> totalVentas = FOREACH ventasGrupo GENERATE COUNT(ventas) AS total;  
grunt> dump totalVentas;
```

1000 ventas totales

```
Util - Total input paths to process : 1  
(all,1000)
```

2. Filtrado por rango de edad

- Filtra los clientes con edad mayor de 30 años y guarda en alias clientes_mayores30.

```
clientes_mayores_30 = FILTER ventas BY edad>30;
```

```
grunt> clientes_mayores_30= FILTER ventas BY edad>30;  
grunt> dump clientes_mayores_30;
```

```
Util - Total input paths to process : 1  
(1,2023-11-24,CUST001,Male,34,Beauty,3,50.0,150.0)  
(3,2023-01-13,CUST003,Male,50,Electronics,1,30.0,30.0)  
(4,2023-05-21,CUST004,Male,37,Clothing,1,500.0,500.0)  
(6,2023-04-25,CUST006,Female,45,Beauty,1,30.0,30.0)
```

- Utiliza LIMIT para ver los primeros 10 resultados.

```
topClientes30 = LIMIT clientes_mayores_30 10;
```

```
grunt> topClientes30 = LIMIT clientes_mayores_30 10;  
grunt> dump topClientes30;
```

```
(1,2023-11-24,CUST001,Male,34,Beauty,3,50.0,150.0)  
(3,2023-01-13,CUST003,Male,50,Electronics,1,30.0,30.0)  
(4,2023-05-21,CUST004,Male,37,Clothing,1,500.0,500.0)  
(6,2023-04-25,CUST006,Female,45,Beauty,1,30.0,30.0)  
(7,2023-03-13,CUST007,Male,46,Clothing,2,25.0,50.0)  
(9,2023-12-13,CUST009,Male,63,Electronics,2,300.0,600.0)  
(10,2023-10-07,CUST010,Female,52,Clothing,4,50.0,200.0)  
(12,2023-10-30,CUST012,Male,35,Beauty,3,25.0,75.0)  
(14,2023-01-17,CUST014,Male,64,Clothing,4,30.0,120.0)  
(15,2023-01-16,CUST015,Female,42,Electronics,4,500.0,2000.0)
```

BIG DATA

- ¿Qué porcentaje del total de transacciones corresponde a clientes mayores de 30?

Grupo clientes mayores 30 (para realizar la operación)

```
clientes30Grupo = GROUP clientes_mayores_30 ALL;
```

Total de clientes mayores de 30

```
totalClientes30 = FOREACH clientes30Grupo GENERATE  
COUNT(clientes_mayores_30) AS mayores30;
```

```
grunt> clientes30Grupo = GROUP clientes_mayores_30 ALL;  
grunt> totalClientes30 = FOREACH clientes30Grupo GENERATE COUNT(clientes_mayores_30) AS mayores30;
```

CROSS → Combinamos los alias totalVentas y totalClientes30 de forma cartesiana. Calculamos el porcentaje multiplicando los mayores de 30 el total de ventas y multiplicando por 100.

```
porcentajeClientes30 = FOREACH (CROSS totalVentas,  
totalClientes30) GENERATE (double)mayores30 / (double)total  
* 100 AS porcentaje_mayores30;
```

```
grunt> porcentajeClientes30 = FOREACH (CROSS totalVentas, totalClientes30) GENERATE (double)mayores30 /  
(double)total * 100 AS porcentaje_mayores30;
```

```
2020-11-17 18:28:00,410 [main] I  
tal input paths to process : 1  
(72.7)
```

3. Transformación de campos

- A partir del conjunto original, crea un alias donde generes

1. el género en mayúsculas (*UPPER(gender)*),

```
ventas_genero_mayus = FOREACH ventas GENERATE ventaID,
fecha, clienteID, UPPER(genero) AS genero_mayusculas, edad,
categoria, cantidad, precioUnidad, precioTotal;
```

```
grunt> ventas_genero_mayus = FOREACH ventas GENERATE ventaID, fecha, clienteID, UPPER(genero) AS genero_mayusculas, edad, categoria, cantidad, precioUnidad, precioTotal;
```

2. una nueva columna *importe_descuento* que calcule, por ejemplo, *price_per_unit * quantity * 0.90* (aplicando un 10% de “descuento ficticio”).

```
ventas_descuento = FOREACH ventas GENERATE ventaID,
fecha, clienteID, UPPER(genero) AS genero_mayusculas, edad,
categoria, cantidad, precioUnidad, precioTotal,
precioUnidad * cantidad * 0.90 AS importe_descuento;
```

```
grunt> ventas_descuento = FOREACH ventas GENERATE ventaID, fecha, clienteID, UPPER(genero) AS genero_mayusculas, edad, categoria, cantidad, precioUnidad, precioTotal, precioUnidad * cantidad * 0.90 AS importe_descuento;
```

- Muestra los primeros 20 registros resultantes.

```
topVentasDescuento = LIMIT ventas_descuento 20;
```

```
grunt> topVentasDescuento = LIMIT ventas_descuento 20;
```

```
(Date, Customer ID, GENDER, Product Category, , , , )
(1, 2023-11-24, CUST001, MALE, 34, Beauty, 3, 50.0, 150.0, 135.0)
(2, 2023-02-27, CUST002, FEMALE, 26, Clothing, 2, 500.0, 1000.0, 900.0)
(3, 2023-01-13, CUST003, MALE, 50, Electronics, 1, 30.0, 30.0, 27.0)
(4, 2023-05-21, CUST004, MALE, 37, Clothing, 1, 500.0, 500.0, 450.0)
(5, 2023-05-06, CUST005, MALE, 30, Beauty, 2, 50.0, 100.0, 90.0)
(6, 2023-04-25, CUST006, FEMALE, 45, Beauty, 1, 30.0, 30.0, 27.0)
(7, 2023-03-13, CUST007, MALE, 46, Clothing, 2, 25.0, 50.0, 45.0)
(8, 2023-02-22, CUST008, MALE, 30, Electronics, 4, 25.0, 100.0, 90.0)
(9, 2023-12-13, CUST009, MALE, 63, Electronics, 2, 300.0, 600.0, 540.0)
(10, 2023-10-07, CUST010, FEMALE, 52, Clothing, 4, 50.0, 200.0, 180.0)
(11, 2023-02-14, CUST011, MALE, 23, Clothing, 2, 50.0, 100.0, 90.0)
(12, 2023-10-30, CUST012, MALE, 35, Beauty, 3, 25.0, 75.0, 67.5)
(13, 2023-08-05, CUST013, MALE, 22, Electronics, 3, 500.0, 1500.0, 1350.0)
(14, 2023-01-17, CUST014, MALE, 64, Clothing, 4, 30.0, 120.0, 108.0)
(15, 2023-01-16, CUST015, FEMALE, 42, Electronics, 4, 500.0, 2000.0, 1800.0)
(16, 2023-02-17, CUST016, MALE, 19, Clothing, 3, 500.0, 1500.0, 1350.0)
(17, 2023-04-22, CUST017, FEMALE, 27, Clothing, 4, 25.0, 100.0, 90.0)
(18, 2023-04-30, CUST018, FEMALE, 47, Electronics, 2, 25.0, 50.0, 45.0)
(19, 2023-09-16, CUST019, FEMALE, 62, Clothing, 2, 25.0, 50.0, 45.0)
```

4. Agrupación y agregación por categoría de producto

- Agrupa por product_category.

```
ventasCategoria = GROUP ventas BY categoria;
```

Para cada categoría calcula: número de transacciones (COUNT), suma de total_amount (SUM), edad promedio de cliente (AVG(age)).

```
resultadosCategoria= FOREACH ventasCategoria GENERATE
group,COUNT(ventas) AS numVentas, SUM(ventas.precioTotal) AS
totalVentas,AVG(ventas.edad) AS mediaEdad;
```

```
grunt> resultadosCategoria= FOREACH ventasCategoria GENERATE group,COUNT(ventas) AS numVentas, SUM(
ventas.precioTotal) AS totalVentas,AVG(ventas.edad) AS mediaEdad;
grunt> dump resultadosCategoria;
```

```
- Total input paths to process : 1
(Beauty,307,143515.0,40.37133550488599)
(Clothing,351,155580.0,41.94871794871795)
(Electronics,342,156905.0,41.73684210526316)
(Product Category,0,,)
grunt>
```

- Ordena el resultado por la suma de total_amount descendente.

```
ordenarResultadosCategoria = ORDER resultadosCategoria BY
totalVentas DESC;
```

```
grunt> describe resultadosCategoria
resultadosCategoria: {group: chararray,numVentas: long,totalVentas: double,mediaEdad: double}
grunt> ordenarResultadosCategoria = ORDER resultadosCategoria BY totalVentas DESC;
```

```
- Total input paths to process : 1
(Electronics,342,156905.0,41.73684210526316)
(Clothing,351,155580.0,41.94871794871795)
(Beauty,307,143515.0,40.37133550488599)
(Product Category,0,,)
```

5. Extracción de categorías distintas

- En este dataset extrae las categorías de producto distintas (DISTINCT product_category).

Recuperar todas las categorías

```
categorias= FOREACH ventas GENERATE categoria;
```

Eliminar filas (categorías) repetidas

```
categoriasDistinct = DISTINCT categorias;
```

```
grunt> categorias = FOREACH ventas GENERATE categoria;
grunt> categoriasDistinct = DISTINCT categorias;
grunt> dump categoriasDistinct;
```

El resultado muestra 3 categorías diferentes, incluyendo el nombre de la columna Product Category.

```
o process : 1
(Beauty)
(Clothing)
(Electronics)
(Product Category)
grunt> |
```

6. Ordenación y obtención de top-transacciones

- Ordena todas las transacciones por total_amount descendente.

```
orderPrecioTotal= ORDER ventas BY precioTotal DESC;
```

- Usa LIMIT para extraer, por ejemplo, las 5 transacciones con mayor total_amount.

```
topPrecioTotal = LIMIT orderPrecioTotal 5;
```

BIG DATA

```
grunt> orderPrecioTotal= ORDER ventas BY precioTotal DESC;  
grunt> topPrecioTotal = LIMIT orderPrecioTotal 5;  
grunt> dump topPrecioTotal;
```

```
(664,2023-12-28,CUST664,Female,44,Clothing,4,500.0,2000.0)  
(742,2023-01-21,CUST742,Female,38,Electronics,4,500.0,2000.0)  
(155,2023-05-17,CUST155,Male,31,Electronics,4,500.0,2000.0)  
(157,2023-06-24,CUST157,Male,62,Electronics,4,500.0,2000.0)  
(572,2023-04-20,CUST572,Male,31,Clothing,4,500.0,2000.0)  
grunt>
```

- Muestra: transaction_id, customer_id, product_category, total_amount.

```
topPrecioTotalSelect = FOREACH topPrecioTotal GENERATE  
ventaID, clienteID,categoria,precioTotal;
```

```
grunt> topPrecioTotalSelect = FOREACH topPrecioTotal GENERATE ventaID, clienteID,categoria,precioTotal;
```

```
(664,CUST664,Clothing,2000.0)  
(742,CUST742,Electronics,2000.0)  
(155,CUST155,Electronics,2000.0)  
(157,CUST157,Electronics,2000.0)  
(572,CUST572,Clothing,2000.0)
```

7. Uso de funciones de cadena

- Añade una nueva columna al alias original donde el product_category se recorte a los primeros 3 caracteres (SUBSTRING(product_category,0,3)) y otra que sea la longitud del product_category (SIZE(product_category)).

```
ventasUpdate= FOREACH ventas GENERATE  
ventaID, fecha,clienteID,genero,edad,categoria,precioUnidad,  
precioTotal,SUBSTRING(categoria,0,3) AS  
categoriaIndice,SIZE(categoria) AS categoriaLen;
```

BIG DATA

```
grunt> ventasUpdate= FOREACH ventas GENERATE ventaID,fecha,clienteID,genero,edad,categoria,precioUnidad,precioTotal,  
SUBSTRING(categoria,0,3) AS categoriaIndice,SIZE(categoria) AS categoriaLen;  
grunt>
```

- Muestra los primeros 15 registros resultantes.

```
topVentasUpdate= LIMIT ventasUpdate 15;
```

```
grunt> topVentasUpdate= LIMIT ventasUpdate 15;  
grunt> dump topVentasUpdate;
```

```
(,Date,Customer ID,Gender,,Product Category,,,Pro,16)  
(1,2023-11-24,CUST001,Male,34,Beauty,50.0,150.0,Bea,6)  
(2,2023-02-27,CUST002,Female,26,Clothing,500.0,1000.0,Clo,8)  
(3,2023-01-13,CUST003,Male,50,Electronics,30.0,30.0,Ele,11)  
(4,2023-05-21,CUST004,Male,37,Clothing,500.0,500.0,Clo,8)  
(5,2023-05-06,CUST005,Male,30,Beauty,50.0,100.0,Bea,6)  
(6,2023-04-25,CUST006,Female,45,Beauty,30.0,30.0,Bea,6)  
(7,2023-03-13,CUST007,Male,46,Clothing,25.0,50.0,Clo,8)  
(8,2023-02-22,CUST008,Male,30,Electronics,25.0,100.0,Ele,11)  
(9,2023-12-13,CUST009,Male,63,Electronics,300.0,600.0,Ele,11)  
(10,2023-10-07,CUST010,Female,52,Clothing,50.0,200.0,Clo,8)  
(11,2023-02-14,CUST011,Male,23,Clothing,50.0,100.0,Clo,8)  
(12,2023-10-30,CUST012,Male,35,Beauty,25.0,75.0,Bea,6)  
(13,2023-08-05,CUST013,Male,22,Electronics,500.0,1500.0,Ele,11)  
(14,2023-01-17,CUST014,Male,64,Clothing,30.0,120.0,Clo,8)
```

8. Filtrado por fecha y condiciones combinadas

- Filtra primero las transacciones que se han hecho antes de una determinada fecha, por ejemplo, `date < '2023-07-01'`. (Suponiendo que el campo `date` es tipo `chararray` con formato `'YYYY-MM-DD'`).

```
fechaFiltro = FILTER ventasUpdate BY fecha<'2023-07-01';
```

```
grunt> fechaFiltro = FILTER ventasUpdate BY fecha<'2023-07-01';  
grunt> dump fechaFiltro;
```

- De ese conjunto, filtra adicionalmente las transacciones con `total_amount > 500`.

```
fechaFiltro500= FILTER fechaFiltro BY precioTotal>500
```

*Se produce casteo de `int` a `double`

BIG DATA

```
hs to process : 1
(2,2023-02-27,CUST002,Female,26,Clothing,500.0,1000.0,Clo,8)
(15,2023-01-16,CUST015,Female,42,Electronics,500.0,2000.0,Ele,11)
(16,2023-02-17,CUST016,Male,19,Clothing,500.0,1500.0,Clo,8)
(31,2023-05-23,CUST031,Male,44,Electronics,300.0,1200.0,Ele,11)
(36,2023-06-24,CUST036,Male,52,Beauty,300.0,900.0,Bea,6)
(42,2023-02-17,CUST042,Male,22,Clothing,300.0,900.0,Clo,8)
(46,2023-06-26,CUST046,Female,20,Electronics,300.0,1200.0,Ele,11)
(48,2023-05-16,CUST048,Male,54,Electronics,300.0,900.0,Ele,11)
(49,2023-01-23,CUST049,Female,54,Electronics,500.0,1000.0,Ele,11)
(54,2023-02-10,CUST054,Female,38,Electronics,500.0,1500.0,Ele,11)
```

- Muestra el resultado, y calcula la edad promedio (AVG(age)) de los clientes que cumplen estas condiciones.

Se agrupa el resultado en un grupo (ALL)

```
fecha500Grupo= GROUP fechaFiltro500 ALL;
```

Se calcula la media del campo edad del grupo

```
edadMediaFecha500= FOREACH fecha500Grupo GENERATE
group,AVG(fechaFiltro500.edad) AS edadMedia;
```

Edad media de los clientes que han comprado antes de la fecha y han gastado más de 500 → 39,33 años

```
hs to process : 1
(all,39.33116883116883)
grunt>
```

9. Script completo + almacenamiento

- Crea un script .pig que contenga los pasos: carga, filtrado, transformación, agrupación, ordenación, y finalmente almacenamiento (STORE) del resultado final

OBJETIVO → Obtener resultados en 2 salidas: los *10 primeros clientes mayores de 30 años con mayor gasto total con IVA en productos de Belleza y total de clientes mayores de 30 años que han comprado productos de Belleza y su gasto medio.*

-- CARGA DATOS

```
ventas = LOAD 'pr03_1/retail_sales_dataset.csv' USING
PigStorage(',') AS (ventaID:int, fecha:chararray,
clienteID:chararray, genero:chararray, edad:int,
categoria:chararray, cantidad:int, precioUnidad:double,
precioTotal:double);
```

-- AÑADIR COLUMNAS IVA (IVA y total conIVA)

```
ventasUpdate = FOREACH ventas GENERATE ventaID, fecha,
clienteID, genero, edad, categoria, cantidad, precioUnidad,
precioTotal, precioTotal * 0.21 AS IVA, precioTotal * 1.21
AS totalIVA;
```

-- FILTER (por categoría Beauty y clientes mayores 30)

```
clientes30BeautyFiltro= FILTER ventasUpdate BY categoria ==
'Beauty' AND edad >30;
```

-- ORDER por totalIVA

```
orderClientes30Beauty = ORDER clientes30BeautyFiltro BY
totalIVA DESC;
```

-- LIMIT 10 primeros registros

```
topClientes30Beauty = LIMIT orderClientes30Beauty 10;
```

BIG DATA

-- GROUP

```
clientes30BeautyGrupo= GROUP clientes30BeautyFiltro ALL;
```

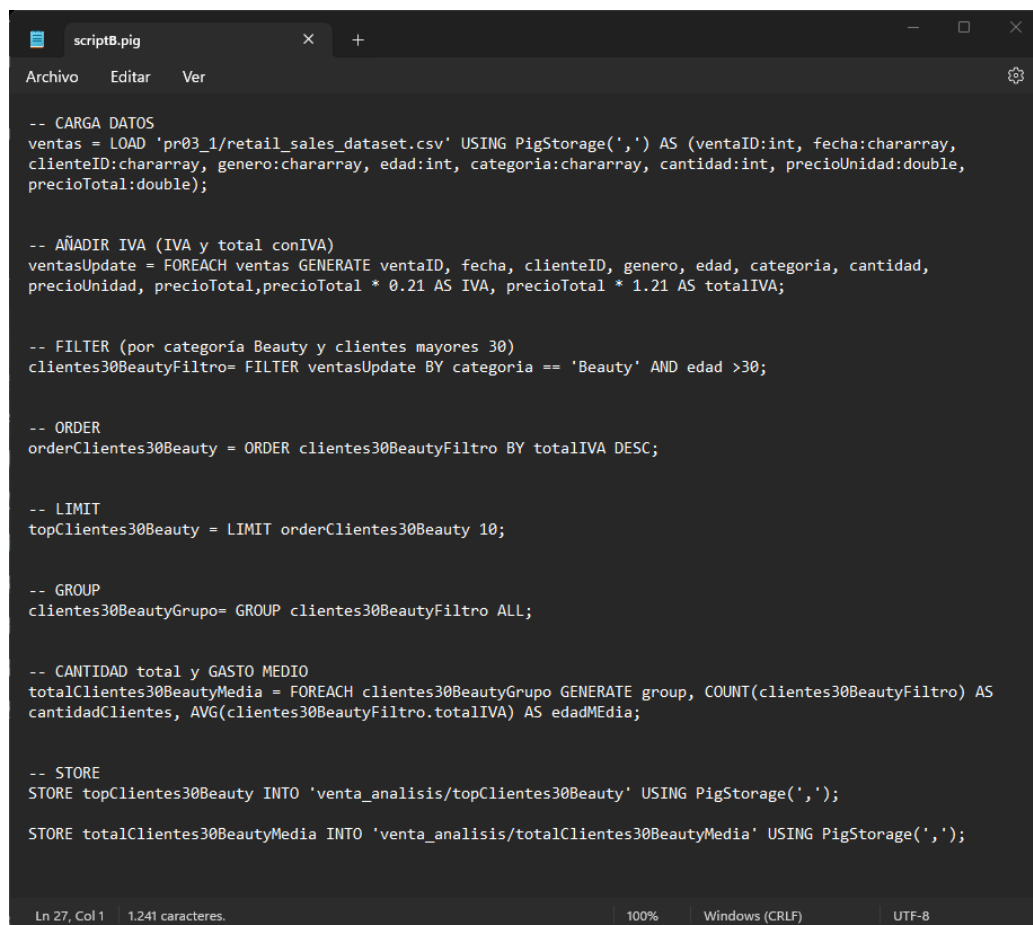
-- CALCULAR CANTIDAD total y GASTO MEDIO

```
resultados = FOREACH clientes30BeautyGrupo GENERATE  
group, COUNT(clientes30BeautyFiltro) AS cantidadClientes,  
AVG(clientes30BeautyFiltro.totalIVA) AS gastoMedio;
```

-- STORE (2 salidas)

```
STORE topClientes30Beauty INTO  
'venta_analisis//topClientes30Beauty' USING  
PigStorage(',');
```

```
STORE totalClientes30BeautyMedia INTO  
'venta_analisis/totalClientes30BeautyMedia' USING  
PigStorage(',');
```



```
scriptB.pig
x +
Archivo Editar Ver
-- CARGA DATOS
ventas = LOAD 'pr03_1/retail_sales_dataset.csv' USING PigStorage(',') AS (ventaID:int, fecha:chararray,
clienteID:chararray, genero:chararray, edad:int, categoria:chararray, cantidad:int, precioUnidad:double,
precioTotal:double);

-- AÑADIR IVA (IVA y total conIVA)
ventasUpdate = FOREACH ventas GENERATE ventaID, fecha, clienteID, genero, edad, categoria, cantidad,
precioUnidad, precioTotal, precioTotal * 0.21 AS IVA, precioTotal * 1.21 AS totalIVA;

-- FILTER (por categoría Beauty y clientes mayores 30)
clientes30BeautyFiltro= FILTER ventasUpdate BY categoria == 'Beauty' AND edad >30;

-- ORDER
orderClientes30Beauty = ORDER clientes30BeautyFiltro BY totalIVA DESC;

-- LIMIT
topClientes30Beauty = LIMIT orderClientes30Beauty 10;

-- GROUP
clientes30BeautyGrupo= GROUP clientes30BeautyFiltro ALL;

-- CANTIDAD total y GASTO MEDIO
totalClientes30BeautyMedia = FOREACH clientes30BeautyGrupo GENERATE group, COUNT(clientes30BeautyFiltro) AS
cantidadClientes, AVG(clientes30BeautyFiltro.totalIVA) AS edadMedia;

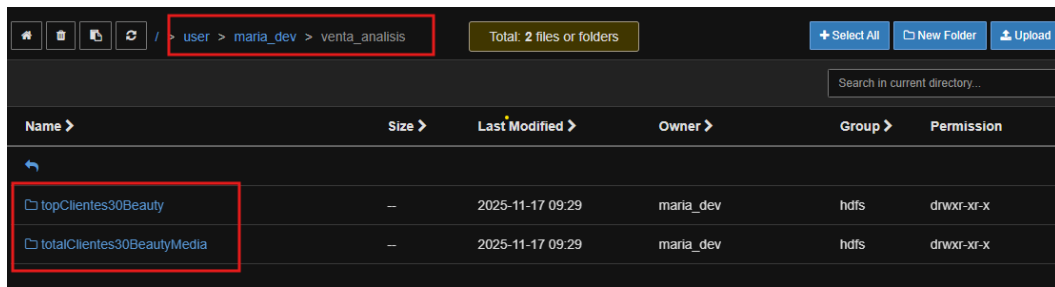
-- STORE
STORE topClientes30Beauty INTO 'venta_analisis/topClientes30Beauty' USING PigStorage(',');

STORE totalClientes30BeautyMedia INTO 'venta_analisis/totalClientes30BeautyMedia' USING PigStorage(',');

Ln 27, Col 1 | 1.241 caracteres. | 100% | Windows (CRLF) | UTF-8
```

BIG DATA

SALIDA DESDE INTERFAZ HDFS de Ambari



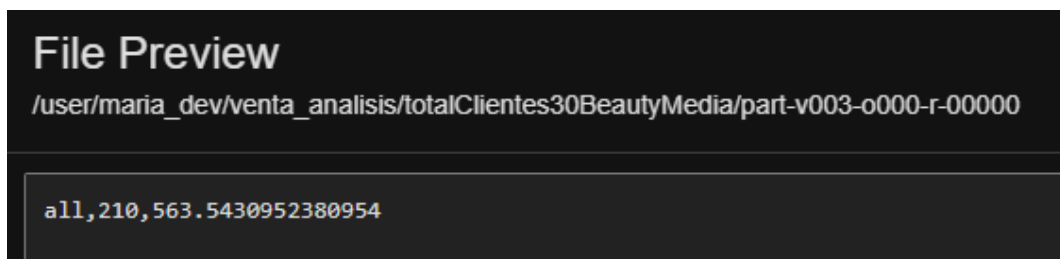
Name >	Size >	Last Modified >	Owner >	Group >	Permission
topClientes30Beauty	—	2025-11-17 09:29	maria_dev	hdfs	drwxr-xr-x
totalClientes30BeautyMedia	—	2025-11-17 09:29	maria_dev	hdfs	drwxr-xr-x

10 primeros clientes mayores de 30 años con mayor gasto total con IVA en productos de Belleza



File Preview									
/user/maria_dev/venta_analisis/topClientes30Beauty/part-v005-o000-r-00000									
503	,2023-10-25	,CUST503	,Male	,45	,Beauty	,4	,500.0	,2000.0	,420.0,2420.0
93	,2023-07-14	,CUST093	,Female	,35	,Beauty	,4	,500.0	,2000.0	,420.0,2420.0
139	,2023-12-15	,CUST139	,Male	,36	,Beauty	,4	,500.0	,2000.0	,420.0,2420.0
832	,2023-09-11	,CUST832	,Male	,47	,Beauty	,4	,500.0	,2000.0	,420.0,2420.0
808	,2023-04-01	,CUST808	,Male	,33	,Beauty	,4	,500.0	,2000.0	,420.0,2420.0
480	,2023-06-29	,CUST480	,Female	,42	,Beauty	,4	,500.0	,2000.0	,420.0,2420.0
743	,2023-01-16	,CUST743	,Female	,34	,Beauty	,4	,500.0	,2000.0	,420.0,2420.0
592	,2023-01-24	,CUST592	,Female	,46	,Beauty	,4	,500.0	,2000.0	,420.0,2420.0
144	,2023-07-15	,CUST144	,Female	,59	,Beauty	,3	,500.0	,1500.0	,315.0,1815.0
164	,2023-05-15	,CUST164	,Female	,47	,Beauty	,3	,500.0	,1500.0	,315.0,1815.0

Total de clientes mayores de 30 años que han comprado productos de Belleza y su gasto medio → 210 clientes y 563,54 de gasto medio



File Preview
/user/maria_dev/venta_analisis/totalClientes30BeautyMedia/part-v003-o000-r-00000
all,210,563.5430952380954

3. APARTADO C

Implementa un contador de palabras (cuantas veces aparece cada palabra)

1. Localiza en Internet una versión del Quijote en formato texto. Descárgala y cópiala a en tu sistema HDFS.

- Implementa en PIG el script necesario para hacer dicha operación.

-- CARGAR

```
quijote = LOAD 'pr03_1/quijote.txt' AS (linea:chararray);
```

-- TOKENIZE → Divide por palabras (tokens) cada línea

```
lineasToken = FOREACH quijote GENERATE TOKENIZE(linea) AS tokensCol;
```

-- FLATTEN → Recoge el comando anterior y convierte cada palabra (token) en un nuevo registro

```
palabraXlinea = FOREACH lineasToken GENERATE FLATTEN(tokensCol) AS palabra;
```

-- GROUP → Agrupa cada valor (palabra) diferente

```
palabraXlineaGrupo = GROUP palabraXlinea BY palabra;
```

-- COUNT → Cuenta nº de palabras por cada grupo

```
totalPalabras = FOREACH palabraXlineaGrupo GENERATE group AS palabra, COUNT(palabraXlinea) AS cantidad;
```

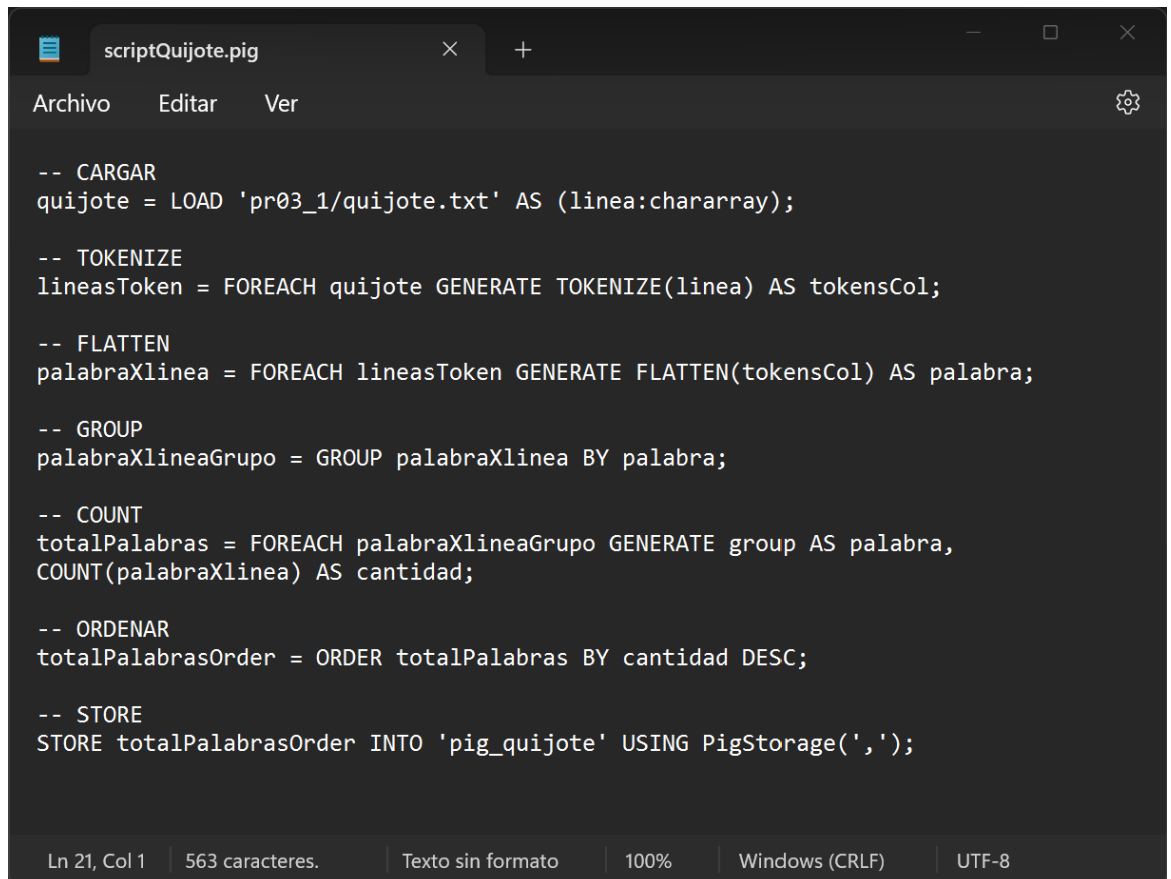
-- ORDENAR

```
totalPalabrasOrder = ORDER totalPalabras BY cantidad DESC;
```

BIG DATA

-- STORE

```
STORE totalPalabrasOrder INTO 'pig_quijote' USING
PigStorage(',');
```



```
scriptQuijote.pig
Archivo  Editar  Ver

-- CARGAR
quijote = LOAD 'pr03_1/quijote.txt' AS (linea:chararray);

-- TOKENIZE
lineasToken = FOREACH quijote GENERATE TOKENIZE(linea) AS tokensCol;

-- FLATTEN
palabraXlinea = FOREACH lineasToken GENERATE FLATTEN(tokensCol) AS palabra;

-- GROUP
palabraXlineaGrupo = GROUP palabraXlinea BY palabra;

-- COUNT
totalPalabras = FOREACH palabraXlineaGrupo GENERATE group AS palabra,
COUNT(palabraXlinea) AS cantidad;

-- ORDENAR
totalPalabrasOrder = ORDER totalPalabras BY cantidad DESC;

-- STORE
STORE totalPalabrasOrder INTO 'pig_quijote' USING PigStorage(',');
```

Ln 21, Col 1 | 563 caracteres. | Texto sin formato | 100% | Windows (CRLF) | UTF-8

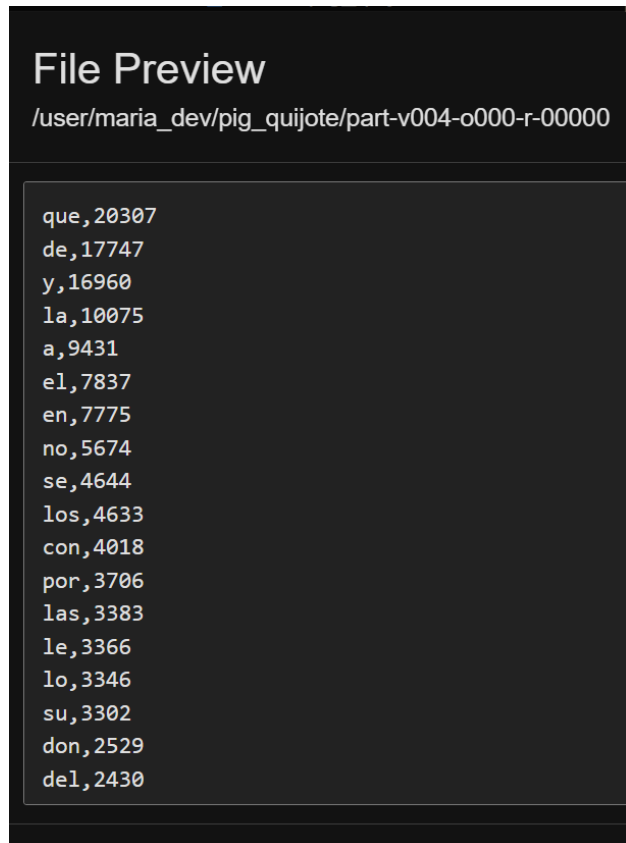
BIG DATA

- Muestra un ejemplo de ejecución sobre El Quijote en pantalla.

Terminal

```
(que,20307)
(de,17747)
(y,16960)
(la,10075)
(a,9431)
(el,7837)
(en,7775)
(no,5674)
(se,4644)
(los,4633)
(con,4018)
(por,3706)
(las,3383)
(le,3366)
(lo,3346)
(su,3302)
(don,2529)
(del,2430)
(me,2324)
(como,2215)
(es,2046)
(más,1964)
(un,1899)
(yo,1815)
(si,1775)
(mi,1665)
(al,1664)
(Sancho,1429)
(Quijote,1413)
```

Interfaz



- Almacena la salida en una carpeta de HDFS llamada /usr/maria_dev/pig_quijote.

